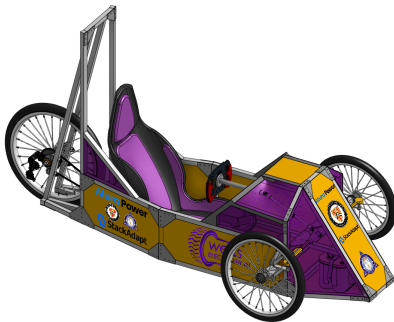


Making a car, then avoiding driving it.

Zane Beeai



June 12, 2025

Outline

- 1 Introduction
- 2 Frame Design
- 3 Front Drivetrain
- 4 Rear Drivetrain
- 5 Electrical System
- 6 Autonomy System

- Minimalist Engineering: simplicity, manufacturability, robustness.
- Quantitative modeling: FEA, drivetrain & electrical analysis.
- Subsystem-based design philosophy.

Design Objectives

- Simplicity Across Subsystems
- DFMA for Manual Tools
- Subsystem Modularity
- Numerical Justification via Simulation
- Electrical Redundancy (telemetry, safeguards)

Material Selection

Aluminum offered ~74% mass reduction compared to steel while maintaining safety margins validated by FEA.

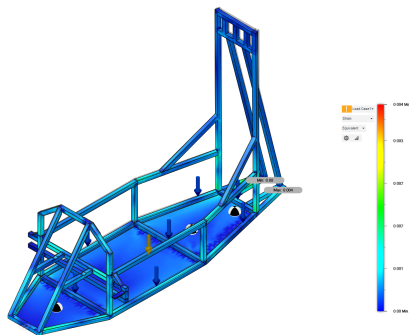


Figure: Figure 1: Frame FEA under 10,000 N load.

Hybrid Assembly

Structural gussets combined with bolts, rivets, and TIG welding reinforced load paths at critical junctions.

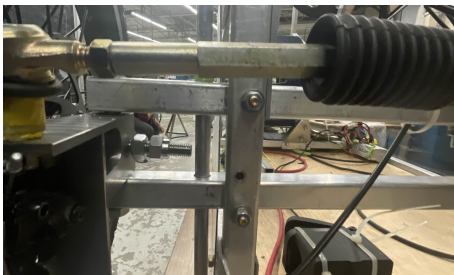


Figure: Figure 2: Modular clearance mount design for track conditions.

Load Distribution

Weight placement balanced the longitudinal COM; battery front-loaded to counter motor mass.

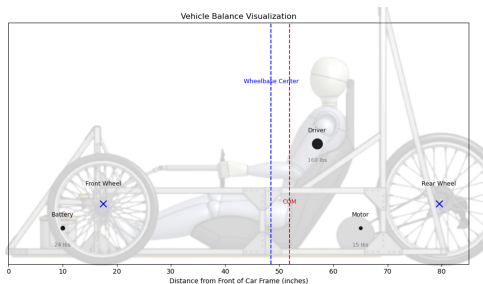


Figure: Figure 3: Longitudinal mass distribution.

FEA Validation

Final frame FEA validated design stability at peak race loads.

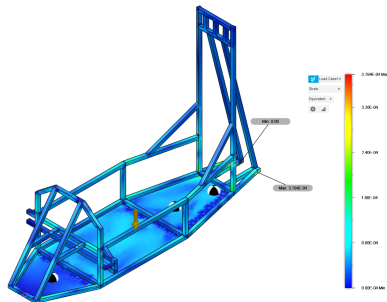


Figure: Figure 4: Von Mises stress simulation.

Stress-Strain Analysis

Strain levels remain far within aluminum elastic region.

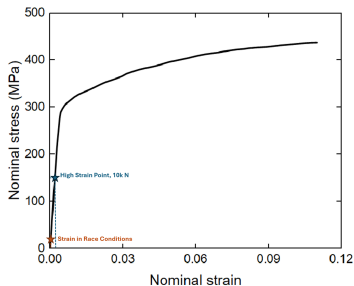


Figure: Figure 5: 1060 aluminum stress-strain curve with FEA data.

Spindle Design

Aluminum spindle failed under braking; steel redesign eliminated excessive deflection.

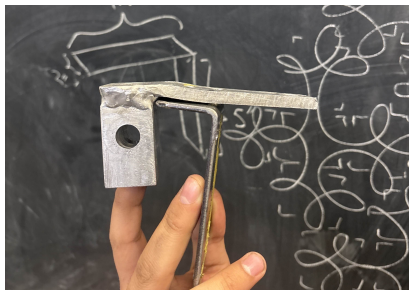


Figure: Figure 6: Aluminum spindle post-failure.

Steel Spindle FEA

1045 steel spindle blocks showed minimal strain under braking simulation.

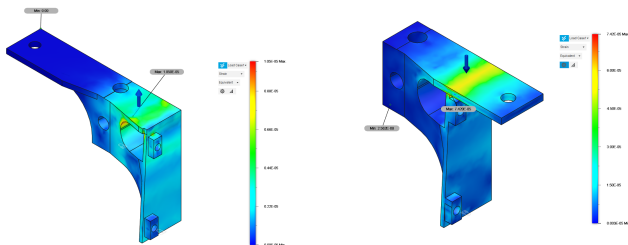


Figure: Figure 7: FEA strain on steel spindles (left/right).

Camber Analysis

Lateral grip maximized near -3° camber (Pacejka-based model).

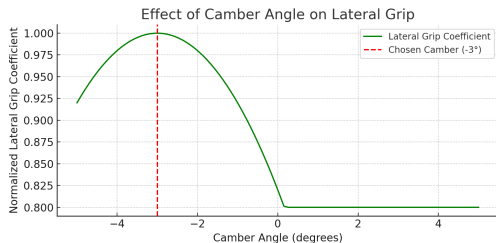


Figure: Figure 8: Camber vs lateral force.

Camber Testing

Visual test confirmed -3° camber in real-world driving.



Figure: Figure 9: Camber confirmation.

Steering Geometry

Ackermann geometry validated for 2.5m turn radius.

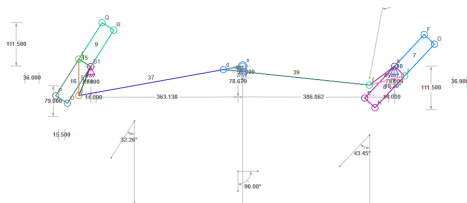


Figure 10: CAD projection (left), drive test (right).

Motor Selection

AmpFlow E30-400-24 provided optimal efficiency at 480W input.

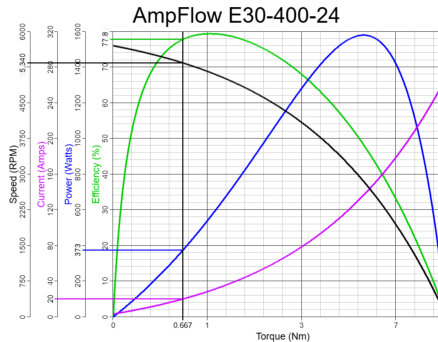


Figure: Figure 11: AmpFlow motor efficiency curve.

Custom Gearbox

Custom 81:16 reduction gearbox minimized sprocket friction losses.

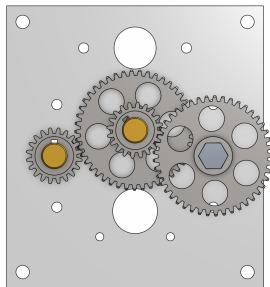


Figure: Figure 12: Custom gearbox design.

Chain Stability

Idler-tensioner reduced chain oscillations and derailments.

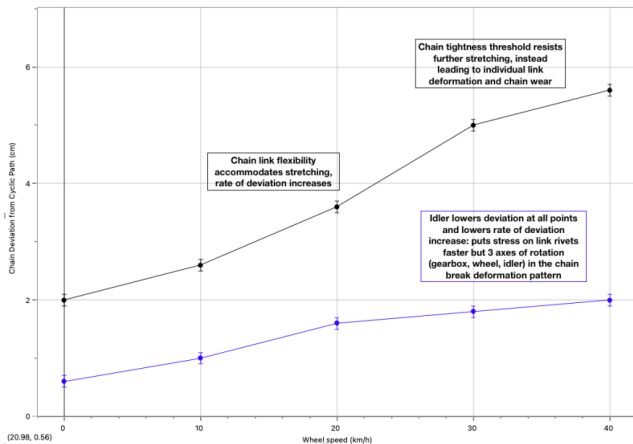


Figure: Figure 13: Chain deviation before/after idler install.

Initial Chain Failure

Chain derailed above 20 km/h prior to idler fix.

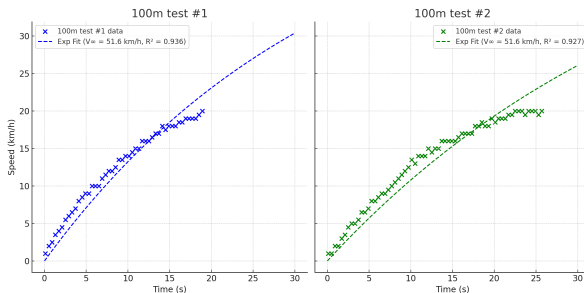


Figure: Figure 14: Pre-fix chain derailment testing.

Acceleration After Fix

Post-fix acceleration stabilized drivetrain operation.

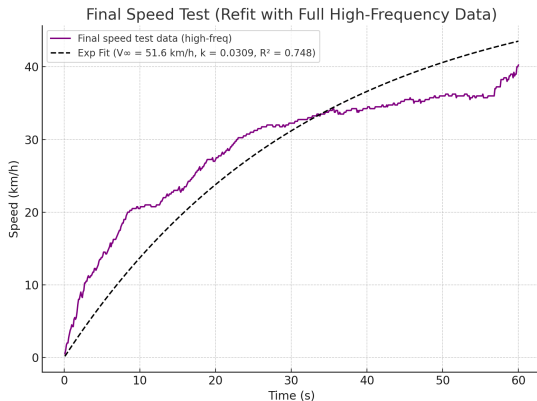


Figure: Figure 15: Final 0–40 km/h acceleration.

Powertrain Architecture

Boosted 12V-24V powertrain maximized efficiency and runtime.

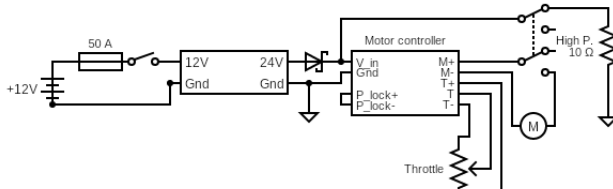


Figure: Figure 16: Full electrical diagram, before self-driving.

Self-Driving Overview

- Fully custom vision-based steering control (camera-only perception).
- Integrated mechanical + electrical + ML stack (data $\rightarrow$ model $\rightarrow$ actuator).
- Three front-mounted 1440p cameras for visual input.
- Control loop: Perception $\rightarrow$ Inference $\rightarrow$ Actuation.

Mechanical Integration

- Motorized steering via gear-driven actuator mounted to the steering column.
- Emergency override kill switch integrated into the main power circuit.
- Arduino Nano provided the low-level interface between motor controller and main compute.



Figure: Figure 17: Motorized steering assembly.

- $3 \times$ 1440p cameras mounted on the front hood.
- USB 3.0 capture to a local compute node.
- Inference computed onboard (CPU/GPU depending on configuration).
- Arduino Nano converts the commanded steering signal to PWM for the steering actuator.

Machine Learning Pipeline

- Dataset generation: manual driving recorded with synchronized images and steering command (label).
- Supervised training on the collected dataset (regression).
- Final model: neural network predicting a continuous steering command θ .
- **Software stack:** OpenCV, TensorFlow/Keras, Python.

Computer Vision Preprocessing

- OpenCV preprocessing used to reduce visual complexity and emphasize lane structure:
 - Edge detection (Canny).
 - Line proposals via HoughLines (optional, for visualization/debugging).
 - Perspective transform to a bird's-eye view (consistent road geometry).
- Output: a compact feature representation (e.g., edge map / warped frame) used as network input.

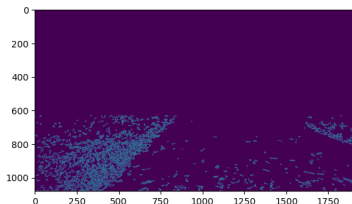


Figure: Figure 19: Sample edge map from preprocessing.

Neural Network Architecture

- Convolutional regression network for behavioral cloning (image $\rightarrow$ steering).
- Layers:
 - Convolution + ReLU + MaxPool $\times 3$
 - Fully connected layers
 - Single output node: predicted steering command θ
- Inspired by end-to-end driving architectures (CNN-based steering regression).

$$\theta = f(\text{image input})$$

Pipeline Validation (MNIST)

- MNIST was used **only** to validate the training codepath (dataloader → model → loss → optimizer).
- Confirmed that training converged and that inference ran correctly on the target compute.
- The driving model was trained on the collected driving dataset (not MNIST).

Behavioral Cloning in Application

- Trained network produced stable steering on the target track conditions represented in the dataset.
- Smooth steering response on both turns and straights after output filtering (deadband / rate limiting).
- Inference latency: < 50 ms (compute only; actuator response is higher).

Video Reference: *Self-Driving Turn Clip.*

Sample Control Output

- Network predicts steering command θ each frame (or at a fixed control rate).
- Post-processing for stability:
 - deadband to suppress small prediction noise
 - saturation to bound maximum steering command
 - optional rate limiting to avoid abrupt steps
- Arduino maps the commanded steering signal to PWM for the actuator.

$$PWM = a \cdot \theta + b$$

Closed-Loop Steering (if angle feedback is available)

- Network outputs θ_{desired} (desired steering angle/command).
- If a steering angle sensor is available, a feedback controller corrects actuator error:

$$e(t) = \theta_{\text{desired}}(t) - \theta_{\text{measured}}(t)$$
$$PWM(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt} + b$$

- Baseline offset b centers the PWM range.
- PID reduces steady-state error and smooths transient response under disturbances/load changes.
- If no angle sensor is present, the system operates open-loop with saturation + rate limiting (no θ_{measured} term).

- Stable control maintained through a straight track segment.
- Self-driving completed both straight and curved segments under the tested conditions.
- Kill-switch handover available in case of system faults.

Video Reference: *Self-Driving Straight Clip.*

Thank You!